

Current CPU Architecture, Data Representation & RISC/CISC

Lesson 59 — von Neumann, Addressing, Hazards, BCD, RISC vs CISC

JKSSB Computer Awareness

October 7, 2026

von Neumann Architecture & the Bottleneck

- ▶ **von Neumann (stored-program) architecture**: instructions and data are held together in the **same memory**.
- ▶ The CPU repeatedly fetches an instruction, decodes it, and executes it, fetching data from the same memory.
- ▶ A single shared path carries both instructions and data.

von Neumann bottleneck

The limited rate at which instructions and data can move between the CPU and memory on that shared path. The CPU often waits for memory, which caps performance.

Addressing Modes: Immediate

- ▶ An **addressing mode** is how an instruction specifies where its operand comes from.
- ▶ **Immediate addressing**: the operand is carried *inside* the instruction itself.
- ▶ No extra memory access is needed to fetch the operand.

Immediate in one line

The operand is part of the instruction, not a separate memory location.

Contrast: register addressing names a register; direct addressing names a memory address.

Cache, Hit Ratio & Why a Hit Is Faster

- ▶ A **cache** is a small, fast memory between the CPU and main memory that holds recently used instructions and data.
- ▶ A **hit** means the requested item is present; a **miss** means it is not.

Cache-hit ratio

$$\text{hit ratio} = \frac{\text{hits}}{\text{hits} + \text{misses}} = \frac{\text{hits}}{\text{total accesses}}.$$

Why a hit is faster

Cache access time is far lower than main-memory access time, so a hit avoids a slow main-memory access.

Data Hazards Even with Register Addressing

- ▶ A **data hazard** is a dependency that stops an instruction from proceeding correctly in the expected cycle.
- ▶ Register operands avoid an extra memory access, but they do **not** remove every hazard.
- ▶ A read-after-write dependency (an instruction needs a register that an earlier instruction is still writing) is a hazard even though no memory operand is involved.

Key point

Hazards come from *dependencies between instructions*, typically exposed by pipelining, not only from memory operands.

Virtual Memory vs Cache

Cache

- ▶ Small, fast memory close to the CPU.
- ▶ Holds recently used blocks.
- ▶ Managed mostly in hardware.
- ▶ Goal: lower average access time.

Virtual memory

- ▶ Makes memory appear larger than physical RAM.
- ▶ Uses secondary storage as backing store.
- ▶ Managed by the OS with hardware help.
- ▶ Goal: run large programs within limited RAM.

The two solve different problems and are not the same hardware.

Instruction Cycle: Program Counter & Instruction Register

- ▶ **Program Counter (PC)**: holds the address of the **next** instruction to be fetched.
- ▶ **Instruction Register (IR)**: holds the instruction currently being fetched, decoded, and executed.
- ▶ After fetching, the PC is advanced to point at the following instruction.

Trap

The PC holds the address of the *next* instruction, **not** the current one being executed.

Interrupts at Instruction Boundaries

- ▶ An **interrupt** is a signal that makes the CPU pause the current program and run a handler.
- ▶ Interrupts are checked at **instruction boundaries**, during the program run.
- ▶ They are **not** delayed until the whole program finishes.

Trap

“Interrupts are processed only after the entire program completes” is false. Handling happens mid-program, between instructions.

Instruction Cycle Counts Vary

- ▶ Different instructions do different amounts of work.
- ▶ A simple register operation may complete in one clock cycle.
- ▶ A divide, a multi-step operation, or an instruction that accesses slow memory can take many cycles.

Trap

“All machine instructions take the same number of clock cycles” is false. The count **varies** by instruction.

Two's-Complement & Overflow Detection

- ▶ In n -bit two's complement the most significant bit is the **sign** bit; negative values are formed by inverting and adding one.
- ▶ **Overflow** means the true result cannot be represented in the available bits.

Overflow test

Overflow occurs when two operands of the **same sign** produce a result of the **opposite sign**.

Trap

Overflow is **not** the same as a carry out of the most significant bit. A carry can occur without any overflow.

Zero-Extension vs Sign-Extension

- ▶ **Extension** widens a value to more bits while keeping its value.
- ▶ **Zero-extension**: fill the new leftmost bits with 0 — used for **unsigned** values.
- ▶ **Sign-extension**: copy the sign bit into the new leftmost bits — used for **signed** values.

Remember the pair

Unsigned → zero-extension; signed → sign-extension. Swapping them is the common trap.

BCD Encoding

- ▶ **BCD** = Binary-Coded Decimal.
- ▶ Each decimal digit is stored in its own **4-bit** code.
- ▶ Decimal 59 becomes 0101 1001 (5 then 9).

Contrast with plain binary

Plain binary of 59 is 111011_2 ; BCD keeps one 4-bit code per decimal digit.

Trap

BCD does **not** pack the whole decimal number as ordinary binary.

Byte-Addressability & Effective Address

- ▶ **Byte-addressable memory**: every individual byte has its own unique address; a word is a group of bytes at consecutive addresses.
- ▶ The **effective address** is the actual location an instruction uses for its operand; it may be computed at run time.
- ▶ Instruction length (in bytes) and effective address are **different** quantities.

Key point

A long instruction does not mean a far address; the operand address is determined by the addressing mode, not by the instruction's size.

RISC vs CISC

Aspect	RISC	CISC
Instruction set	Small, simple	Large, complex
Instruction length	Fixed	Variable
Registers	Many	Fewer
Memory access	Load/store only	Memory operands allowed
Cycles per instruction	Typically one (pipelined)	Often multiple
Control emphasis	Hardwired, compiler-driven	Microcoded, hardware-driven

RISC = Reduced Instruction Set Computer; CISC = Complex Instruction Set Computer.

Lesson 59: Recall

- ▶ von Neumann: instructions and data share one memory; the shared path causes the von Neumann bottleneck.
- ▶ Immediate addressing keeps the operand inside the instruction.
- ▶ Cache-hit ratio = hits / total accesses; a hit is faster than main memory.
- ▶ Hazards come from dependencies, even with register addressing.
- ▶ The PC holds the address of the **next** instruction; the IR holds the current instruction.
- ▶ Interrupts are handled at instruction boundaries, mid-program.
- ▶ Clock-cycle counts vary by instruction.
- ▶ Overflow = same-sign operands give an opposite-sign result.
- ▶ Zero-extension for unsigned; sign-extension for signed.
- ▶ BCD uses one 4-bit code per decimal digit.
- ▶ RISC = simple fixed-length, many registers, load/store.